MOMA METRICS UPLOAD UTILITY

Technical Reference Document

EMEA Non-Production Environment

Script Name	MOMA_METRICS.ps1
Version	1.0
Author	Mohan Kumar N
Approver / Team Lead	Romain GROSDEMANGE
Team	EMEA L4 DBA
Created On	17-Mar-2026
Document Date	22-May-2026
Environment	EMEA non-production
Location Tag	EMEA

1. Overview

MOMA_METRICS.ps1 is a PowerShell utility developed by the EMEA L4 DBA team to collect health and performance metrics from one or more Microsoft SQL Server instances and push them to MOMA — an internal Prometheus-compatible monitoring platform (push gateway). The script translates SQL Server DMV data into the Prometheus exposition format and uploads it via HTTP POST, enabling real-time alerting and dashboarding in MOMA.

1.1 Purpose

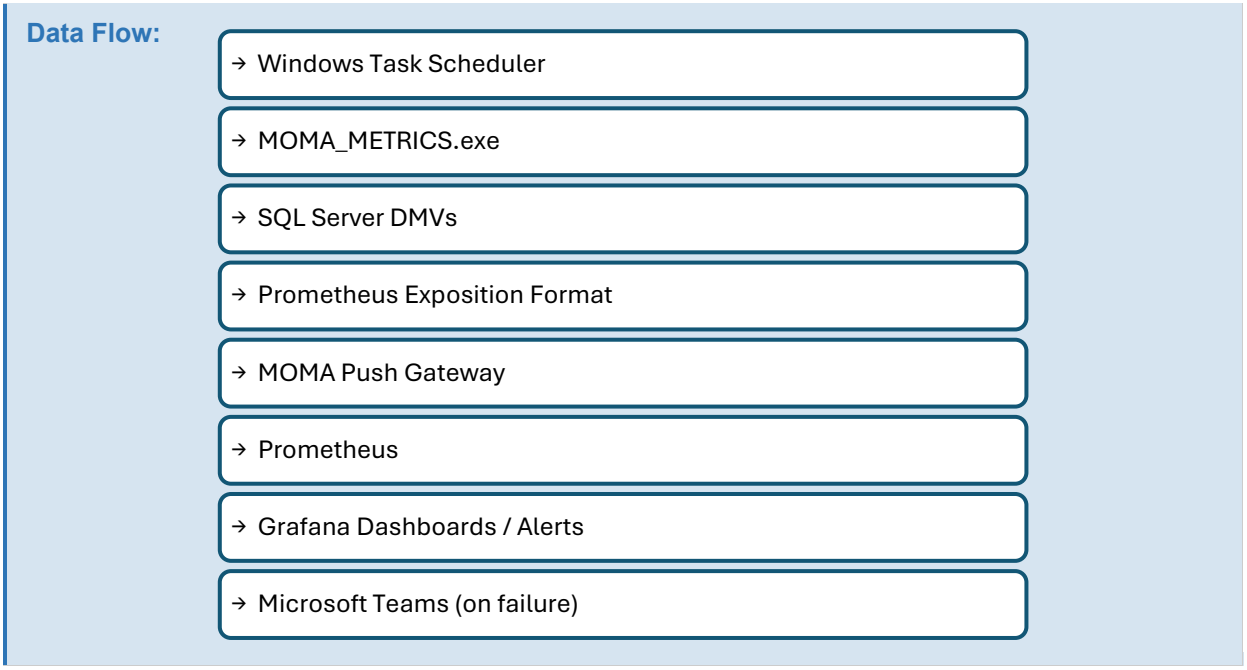
- Centralise SQL Server observability into the MOMA / Prometheus stack without installing agents on every SQL host.
- Provide consistent, labelled metrics across all EMEA Non-Production instances.
- Alert the DBA team via Microsoft Teams when data collection or upload fails.

1.2 Key Design Principles

- Config-driven: every metric category is toggled from a database table (moma.MOMA_Config), so no script changes are needed to enable or disable a metric.
- Schedule-aware: the $Schedule parameter controls which metric frequency group runs, allowing the Windows Task Scheduler to call the script at different intervals.
- Multi-server: a comma-separated server list is accepted, and the script loops through each server independently.
- Encrypted credentials: SQL login credentials are stored as DPAPI-encrypted strings — they are machine-bound and cannot be decrypted on another host.

2. Architecture & Data Flow

The diagram below illustrates the end-to-end data flow of the script:



Each execution cycle proceeds through the following high-level stages:

- Parameter intake and validation
- Log file maintenance
- Credential decryption and database connection
- Configuration load from moma.MOMA_Config
- Dynamic query list construction
- SQL metric collection via DMVs
- Metric upload to MOMA push gateway
- Error alerting to Microsoft Teams webhook

3. Pre-Requisites

3.1 Software Requirements

- PowerShell 5.1 or PowerShell 7.x (Windows)
- SqlServer PowerShell module (provides Invoke-Sqlcmd) — install with: Install-Module -Name SqlServer
- Network access to the MOMA push gateway endpoint: https://in-moma-prom-ppr00.essilor.cloud
- Network access to the Power Automate webhook endpoint for Teams alerting

3.2 SQL Server Requirements

- The SQL login whose credentials are embedded in the script must have VIEW SERVER STATE and VIEW DATABASE STATE permissions.
- The XEvent session DBA_XMLDeadlockReport must exist and be in a running state for deadlock metrics to populate.
- Log-shipping monitoring tables (msdb.dbo.log_shipping_monitor_*) must be present if log-shipping metrics are enabled.
- The distribution database must exist if replication metrics are enabled (the script gracefully skips if absent).
- The configuration database (ITE_DBA_ADMIN_DB or AdminSQL) must contain the schema moma and table MOMA_Config.

3.3 MOMA_Config Table Schema

The script reads its operating parameters from the following table on the target SQL instance:

Column	Type	Description
MetCat	VARCHAR	Metric category key — matches QueryMap keys in the script
Threshold	INT	Alert threshold value used by MOMA / Prometheus alerting rules
MomaAlert	BIT	1 = metric is active and should be collected; 0 = skip
Frequency	INT	Polling interval in minutes (5, 20, 30, or 1440)

Note: The script tries ITE_DBA_ADMIN_DB first, then AdminSQL, then falls back to master. Ensure MOMA_Config exists in whichever database is accessible.

4. Script Parameters

4.1 Parameter Reference

Parameter	Type	Valid Values	Description
-Schedule	INT	5, 20, 30, 1440	Polling frequency in minutes. Filters MOMA_Config rows and determines which metric group to execute. Default: 30.
-Server	STRING[]	Hostname / FQDN	One or more SQL Server host names (comma-separated). Required.

4.2 Schedule Values

Value	Interval	Typical Use
5	Every 5 minutes	Near real-time checks (blocking, deadlocks, CPU)
20	Every 20 minutes	Medium-frequency checks (AG health, replication)
30	Every 30 minutes	Standard checks (database space, disk, memory, jobs)
1440	Once per day	Daily checks (backup age, log-shipping thresholds)

4.3 Invocation Examples

Single server, 30-minute schedule:

```
.\MOMA_METRICS.ps1 -Schedule 30 -Server SQLSERVER01
```

Multiple servers, 5-minute schedule:

```
.\MOMA_METRICS.ps1 -Schedule 5 -Server SQLSERVER01,SQLSERVER02,SQLSERVER03
```

FQDN server, daily schedule:

```
.\MOMA_METRICS.ps1 -Schedule 1440 -Server FRER1932.europe.essilor.group
```

5. Metric Categories

The table below documents every metric category the script can collect, together with the Prometheus metric name emitted and the SQL source used.

#	Metric Category	Prometheus Metric Name	Description	Schedule
1	DatabaseStatus	mssql_dba_db_status_ratio	Online/Offline status of every database (1 = online, 0 = offline)	Any
2	DatabaseSpace_Used	mssql_dba_database_useddbsize_size_bytes	Actual used data & log space in bytes	Any
3	DatabaseSpace_Total	mssql_dba_database_totaldbsize_size_bytes	Total allocated file size in bytes	Any
4	DatabaseSpace_Max	mssql_dba_database_maxdbsize_size_bytes	Max allowable size (disk-free used when file is unlimited)	Any
5	Blocking	mssql_dba_custom_blocking_count	Number of sessions blocked for ≥ 20 seconds	Any
6	Deadlock	mssql_dba_custom_deadlock_count	Deadlock events in last 5 minutes via XEvent session DBA_XMLDeadlockReport	Any
7	DiskSpace	mssql_dba_diskspace_used_percent	Disk usage % per drive hosting SQL files	Any
8	CPU	mssql_dba_cpu_percent_maximum_Percent	SQL Server process CPU utilisation % (ring buffer)	Any
9	Memory	mssql_dba_memory_sqlserver_percent	SQL Server physical memory usage % of total host RAM	Any
10	JobFail	mssql_dba_custom_agent_Jobfailed_result	Enabled SQL Agent jobs whose last run status is failed	Any
11	AlwaysOnHealth	mssql_dba_aag_health	AG synchronisation health and role (Primary/Secondary)	Any
12	AutoGrowthEvent_Row	mssql_dba_autogrowth_row_event	Data-file auto-growth events captured via XEvent	Any
13	AutoGrowthEvent_Log	mssql_dba_autogrowth_log_event	Log-file auto-growth events captured via XEvent	Any
14	DatabaseBackup	mssql_dba_database_backup_*	Last backup age and type for each database	Any
15	Logshipping_Restore_Latency	mssql_dba_logshipping_restore_latency / _threshold	Restore latency & threshold (minutes) per secondary DB	Any
16	Logshipping_Backup_Latency	mssql_dba_logshipping_backup_latency / _threshold	Backup age (minutes since last backup) & threshold per primary DB	Any
17	Logshipping_Copy_Latency	mssql_dba_logshipping_copy_latency / _threshold	Copy lag (minutes since last copy) & threshold per secondary DB	Any
18	Replication	mssql_dba_replication_latency	Transactional replication latency (seconds) per publication	Any

Label Convention: Every metric carries the labels: host_name, instance_name, location=EMEA, plus category-specific labels such as db, type, Drive_name, database_name, publication, etc. Labels are formatted using the Prometheus exposition syntax: metric_name{label1="value1", ...} numeric_value

6. Step-by-Step Execution Flow

Step 1 — Parameter Validation

The script validates that both -Schedule and -Server are provided and non-empty. If either is missing, a red error message is printed and the script exits immediately.

Step 2 — Log File Maintenance

A log file named MOMAUPLOADED_<Schedule>.LOG is maintained in the script's working directory. Before each run, the file size is checked: if it exceeds 10 MB, the file is deleted and a fresh log is started. This prevents unbounded disk growth on long-running Task Scheduler deployments.

Step 3 — Server Loop Initialisation

The -Server value is split on commas to produce a list of target server names. The outer loop iterates once per server. All subsequent steps occur within this loop.

Step 4 — Credential Decryption & Connection String Build

SQL credentials are stored as DPAPI-encrypted hex strings directly in the script body. At runtime they are decrypted using ConvertTo-SecureString and [System.Net.NetworkCredential]. The script then attempts to connect to ITE_DBA_ADMIN_DB, then AdminSQL, then falls back to master. The first successful connection is used for all subsequent queries against that server.

> **Security:** DPAPI encryption is machine-bound. The encrypted strings ($ep) will only decrypt correctly on the Windows host on which they were originally encrypted. Re-encryption is required when the script is moved to a new host.

Step 5 — Configuration Load from MOMA_Config

The query below is executed against the chosen database:

```
SELECT MetCat, Threshold, MomaAlert, Frequency FROM moma.MOMA_Config WHERE
Frequency = <Schedule>
```

Results are loaded into an in-memory hashtable ($ConfigMap) keyed by MetCat. Only rows matching the current -Schedule value are loaded.

Step 6 — Dynamic Query List Construction

An internal $QueryMap hashtable maps every MetCat key to one or more SQL query strings. The script iterates over this map and adds a query to the active $Queries list only if the corresponding MOMA_Config row has MomaAlert = 1. This means disabling a metric in the database table is sufficient — no script edits are required.

A safety check then verifies that at least one query was enabled. If $Queries is empty, an exception is thrown to prevent a silent no-op run.

Step 7 — Metric Collection & Upload

For each query in $Queries:

- Invoke-Sqlcmd executes the query using the connection string built in Step 4.
- The Text column of the result set (the Prometheus-formatted string) is extracted.
- Upload-Metrics posts the payload to the MOMA push gateway via HTTP POST with Content-Type: text/plain.
- The raw metric strings are appended to the log file for audit purposes.
- Send-TeamStatus posts a status notification to the Teams webhook (success path).

If an individual query fails, the error is logged, a Teams alert is sent via Send-ErrorAlert, and the loop continues with the remaining queries. After all queries are processed, if any query failed, an exception is raised so the outer error handler captures the overall failure.

Step 8 — Error Alerting

Send-ErrorAlert constructs a JSON body containing the app name (server), the error message, and the current timestamp, then posts it to the Power Automate webhook ($ErrorWebhook). The Power Automate flow routes this to the designated EMEA DBA Teams channel.

Step 9 — Log Footer

A final block appends an END marker to the log file regardless of whether the run succeeded or failed. This allows easy log parsing to identify incomplete runs.

7. SQL Query Details

7.1 Database Size (mssql_dba_database_*dbsize_size_bytes)

This is the most complex query. It uses five CTEs to compute used, total, and maximum size for both data (ROWS) and log (LOG) files per database, then cross-joins with a Metrics CTE to emit three metric rows per database per file type. Key details:

- VolumeInfo: gets available disk bytes per file type using sys.dm_os_volume_stats.
- DataFileSizes / LogFileSizes: computes page counts from sys.master_files joined to sys.dm_db_file_space_usage (data) and sys.dm_db_log_space_usage (log).
- When a file has unlimited growth (max_size = -1), MaxSizeBytes is calculated as current file size + available disk space rather than a fixed ceiling.
- All sizes are converted from SQL pages (8192 bytes each) to bytes for Prometheus.

7.2 Database Status (mssql_dba_db_status_ratio)

Reads sys.databases with NOLOCK. Emits 1 for state = 0 (online) and 0 for any other state. Includes instance name construction with backslash separator for named instances.

7.3 Blocking (mssql_dba_custom_blocking_count)

Uses two CTEs: HeadBlocker (finds the session that is blocking others with wait_time >= 20,000 ms) and HeadBlockerSQL (retrieves the running T-SQL statement). The blocking count is left-joined with the deadlock XEvent data so both metrics can be emitted together.

7.4 Deadlocks (mssql_dba_custom_deadlock_count)

Reads from the DBA_XMLDeadlockReport XEvent session file target. Uses sys.fn_xe_file_target_read_file to parse .xel files and counts events within the last 5 minutes. Returns the deadlock XML payload for diagnostic logging as well as the numeric count for Prometheus.

7.5 Disk Space (mssql_dba_diskspace_used_percent)

Uses sys.dm_os_volume_stats via CROSS APPLY on sys.master_files to get available_bytes and total_bytes per mount point. Emits used percentage (100 minus free percentage) per drive letter.

7.6 CPU (mssql_dba_cpu_percent_maximum_Percent)

Reads the RING_BUFFER_SCHEDULER_MONITOR ring buffer from sys.dm_os_ring_buffers and parses the XML record to extract SQLProcessUtilization (SQL Server CPU %). Returns the most recent sample only (TOP 1 ordered by timestamp DESC).

7.7 Memory (mssql_dba_memory_sqlserver_percent)

Divides physical_memory_in_use_kb from sys.dm_os_process_memory by total_physical_memory_kb from sys.dm_os_sys_memory to compute the percentage of host RAM consumed by the SQL Server process.

7.8 SQL Agent Jobs (mssql_dba_custom_agent_Jobfailed_result)

Queries msdb.dbo.sysjobs joined to sysjobschedules and sysjobhistory. Returns only enabled jobs whose most recent step execution (step_id <> 0) has run_status = 0 (failed). The DBA_SystemEventAlert job is explicitly excluded to avoid false alerts from internal maintenance jobs.

7.9 Always On Availability Groups (mssql_dba_aag_health)

Queries sys.dm_hadr_availability_replica_states and related DMVs to report AG synchronisation health, replica role, and operational state per replica. Non-applicable on instances without AG configured.

7.10 Log Shipping (mssql_dba_logshipping_*)

Six separate queries cover the three log-shipping stages (backup, copy, restore) for both current latency and configured threshold values:

- Restore latency / threshold: msdb.dbo.log_shipping_monitor_secondary
- Backup latency / threshold: msdb.dbo.log_shipping_monitor_primary
- Copy latency / threshold: calculated as DATEDIFF(MINUTE, last_copied_date / last_restored_date, GETDATE())

7.11 Replication (mssql_dba_replication_latency)

Checks for the distribution database using DB_ID('distribution'). If present, queries distribution.dbo.MSrepl_transactions joined to MSdistribution_agents to calculate seconds elapsed since the last replicated transaction per publication. Gracefully prints a message and emits no rows if distribution is absent.

8. Logging

Every execution appends structured entries to MOMAUPLOADED_<Schedule>.LOG in the script's working directory. The log structure is:

```
START----------------------
<timestamp: yyyy-MM-dd HH:mm:ss>
<ServerName>
<Schedule value>
<raw Prometheus metric lines>
END----------------------
```

If a query fails, the error message is written between the START and END markers. Log entries from multiple servers appear sequentially. The log file is automatically deleted and recreated when it exceeds 10 MB.

9. Error Handling & Alerting

9.1 Query-Level Errors

Each SQL query is wrapped in its own try/catch block. A failure sets a $QueryFailed flag, logs the error message, and calls Send-ErrorAlert. Execution continues with the remaining queries so a single failing metric does not suppress all others.

9.2 Fatal Errors

Failures at the connection or configuration-load stage (before metric collection begins) cause an immediate exception. $ErrorActionPreference is set to Stop so uncaught exceptions terminate the script. The Teams webhook receives an alert in either case.

9.3 Teams Webhook Alerts

Both Send-ErrorAlert and Send-TeamStatus post JSON payloads to a Power Automate HTTP trigger. The payload includes app name (server), error message (on failure), and a timestamp. The flow routes alerts to the EMEA DBA Teams channel.

10. Troubleshooting Guide

Symptom	Likely Cause	Resolution
No queries enabled for schedule X	No rows in moma.MOMA_Config with matching Frequency and MomaAlert=1	Insert/update the config table for that schedule
Upload fails with HTTP error	MOMA Prometheus push-gateway unreachable or endpoint URL changed	Verify $MomaApiUrl and network connectivity from the host
Invoke-Sqlcmd login failure	Encrypted credentials ($ep) were generated on a different host	Re-encrypt credentials using DPAPI on the target host
Deadlock query returns no rows	XEvent session DBA_XMLDeadlockReport is not running	Create/start the XEvent session on the SQL instance
Log-shipping query returns no rows	Instance is neither a primary nor secondary in log-shipping	Verify log-shipping is configured; query can safely return 0 rows
Replication query returns no rows	Distribution database does not exist on the instance	Script handles gracefully with PRINT; no action needed
Teams alert not sent	Power Automate webhook URL is invalid or flow is disabled	Validate $ErrorWebhook URL and check the Power Automate flow
Log file exceeds 10 MB	Script auto-removes the log and starts fresh	No action required; reduce $Schedule frequency if needed

11. Task Scheduler Configuration

The script is designed to be invoked by Windows Task Scheduler with one task per schedule frequency. A typical configuration is:

Setting	Recommended Value
Program/Script	powershell.exe
Arguments (5-min)	-ExecutionPolicy Bypass -File C:\DBA\MOMA_METRICS.ps1 -Schedule 5 -Server SERVER01,SERVER02
Arguments (30-min)	-ExecutionPolicy Bypass -File C:\DBA\MOMA_METRICS.ps1 -Schedule 30 -Server SERVER01,SERVER02
Arguments (daily)	-ExecutionPolicy Bypass -File C:\DBA\MOMA_METRICS.ps1 -Schedule 1440 -Server SERVER01
Run As	Local service account or domain account with VIEW SERVER STATE on target instances
Run Whether User Logged On	Yes
Trigger (5-min task)	Repeat every 5 minutes indefinitely
Trigger (30-min task)	Repeat every 30 minutes indefinitely

Important: Because DPAPI credentials are machine-bound, the script must run on the same Windows host on which the encrypted strings were originally generated. Do not copy the script to a different machine without re-encrypting the credentials.

12. Maintenance & Operational Notes

- Enabling/disabling a metric: update MomaAlert in moma.MOMA_Config — no script change required.
- Adding a new server: add it to the -Server argument in the Task Scheduler task definition.
- Changing thresholds: update the Threshold column in moma.MOMA_Config; these values are available to MOMA/Prometheus alerting rules but are not evaluated by the script itself.
- Credential rotation: re-encrypt the new password using ConvertFrom-SecureString on the target host and update the $ep variables in the script.
- Version control: the script header contains Created On and version details. Update these on every change and commit the updated file to the team repository.
- Log review: periodically review MOMAUPLOADED_*.LOG files for repeated error entries that may indicate a degraded SQL instance or network connectivity issue.

13. Security Considerations

- Credentials: DPAPI-encrypted strings are tied to the Windows machine account. Physical or virtual machine access is required to decrypt them — they cannot be extracted by reading the script file alone.
- Network: all metric data is transmitted over HTTPS to the MOMA push gateway. Ensure TLS certificates are valid and trusted on the host.
- Permissions: the SQL login requires only read-level DMV permissions (VIEW SERVER STATE, VIEW DATABASE STATE). It should not have DDL or DML rights.
- Webhook URLs: the Power Automate webhook URL embedded in the script contains a shared access signature. Treat it as a secret and rotate it if the script is ever exposed outside the DBA team.
- Log content: metric payloads written to the log file contain database names, file paths, and performance counters. Ensure the log directory has appropriate NTFS ACLs.

14. Revision History

Version	Date	Author	Change Summary
1.0	17-Mar-2026	Mohan Kumar N	Initial release — all 18 metric categories, DPAPI credentials, Teams alerting
1.1	28-Mar-2026	EMEA L4 DBA	Version banner updated; Schedule 20 added to ValidateSet